

Formation Spring

Configuration par classes annotées



Prise en main de Spring via classes annotées en @Configuration

2

- ✓ Classe annotée
- ✓ Déclaration et paramétrage des beans
- ✓ Création du contexte Spring
- ✓ Récupération d'un bean
- ✓ Scope
- ✓ Properties

Paramétrage par classe annotée

- Existe depuis la version 3 du Spring
- A commencé à être utilisé vers le début du Spring 4
- Est toujours utilisé aujourd'hui mais souvent en complément de Spring Boot
 - Spring Boot est une évolution des classes annotées en `@Configuration`

Paramétrage par classe annotée

➤ **Avantages :**

- Tout via le code Java et les annotations du Spring
- Peut s'explorer en plusieurs classes (rangées par thèmes métiers ou fonctionnelles)
- Pas de limitation (set/constructeur) contrairement à l'XML

➤ **Inconvénients :**

- Si modifiée : besoin de recompiler le code
- Besoin de connaître les annotations du Spring associées à la configuration
- Parfois assez lourd selon les modules du Spring

Définition Simple - Classe annotée

- Une classe de configuration Spring sera annotée par `@org.springframework.context.annotation.Configuration`
- Elle pourra contenir d'autres annotations de configuration en fonction des besoins et/ou des modules Spring utilisés
- Dans certains cas, pour certains modules, la classe ne sera pas annotée mais héritera d'une classe particulière du Spring
 - Par exemple en Spring MVC

Définition Simple - Classe annotée

- L'annotation `@org.springframework.context.annotation.Bean` se place sur une méthode
- Elle permet de définir un bean, au sens Spring = géré par le contexte du Spring.
- C'est **VOUS** qui devez instancier vos objets dans les méthodes annotées en `@Bean`.
- C'est le **SPRING** qui se charge de laisser le bean à disposition ainsi fabriqué pour votre code ou vos autres `@Bean`

Définition Simple - Classe annotée

➤ Exemple

```
package com.exo;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SpringConfiguration {

    @Bean
    public Client client1() {
        // C'est à VOUS de fabriquer vos objets
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }
}
```

Un bean de type
Client et d'id "client1"
est placé dans le
contexte Spring

Définition Simple - Classe annotée

- Par défaut, le nom de la méthode est l'id du bean
- Dans l'exemple précédent, le bean de type *Client* est disponible sous le nom "client1".
- Vous pouvez aussi faire usage de l'attribut **name** de l'annotation `@Bean` si vous préférez.

```
@Configuration
public class SpringConfiguration {
    @Bean(name="client1")
    public Client tarladada() {
        ...
    }
}
```

Un bean de type *Client* et d'id "client1" est placé dans le contexte Spring

Définition Simple - Classe annotée

- Les liens **entre beans** se feront naturellement via les liens entre méthodes annotées **@Bean**

```
@Configuration
public class SpringConfiguration {

    @Bean
    public Client client1() {
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }

    @Bean
    public List<Client> clients() {
        List<Client> resu = new ArrayList<>();
        // On appelle la méthode annotée
        resu.add(this.client1());
        return resu;
    }
}
```



Définition Simple - Classe annotée

- Les liens **entre bean** peuvent aussi se faire via les paramètres de vos méthodes annotées
- Ainsi, le Spring peut y injecter des @Bean connus
- Cette injection se fera par
 - **Nom** du paramètre
 - **Typage** du paramètre
 - Utilisation de l'annotation **@Qualifier** si ambiguïté

Définition Simple - Classe annotée

- Exemple par **nom de paramètre**
- Il y a ici deux *@Bean* de type Client
- Le Spring regarde
 1. Le typage
 2. Puis le nom du paramètre

```
@Configuration
public class SpringConfiguration {

    @Bean
    public Client client1() {
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }

    @Bean
    public List<Client> clients(Client client1) {
        List<Client> resu = new ArrayList<>();
        resu.add(client1);
        return resu;
    }

    @Bean
    public Client client2() {
        Client resu = new Client();
        resu.setNom("Durand");
        resu.setPrenom("Alfred");
        resu.setAge(12);
        return resu;
    }
}
```

Définition Simple - Classe annotée

➤ Exemple par le **type**

➤ Un seul **@Bean** de type **Client**

- Si plusieurs, erreurs du Spring au chargement du contexte

❑ `org.springframework.beans.factory.UnsatisfiedDependencyException`

```
@Configuration
public class SpringConfiguration {

    @Bean
    public Client client1() {
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }

    @Bean
    public List<Client> clients(Client pC) {
        List<Client> resu = new ArrayList<>();
        resu.add(pC);
        return resu;
    }
}
```

Définition Simple - Classe annotée

13

➤ Exemple par l'utilisation de **@Qualifier**

➤ Prend le name (ou l'id) du bean en paramètre, peut-être

- ❑ le nom de la méthode
- ❑ la valeur de name de @Bean

➤ A réserver dans les cas ambigus

```
@Configuration
public class SpringConfiguration {

    @Bean
    public Client client1() {
        Client resu = new Client();
        resu.setNom("Dupont");
        return resu;
    }

    @Bean
    public List<Client> clients(@Qualifier("client1") Client pC, @Qualifier("cl2") Client pC2) {
        List<Client> resu = new ArrayList<>();
        resu.add(pC);
        resu.add(pC2);
        return resu;
    }

    @Bean(name="cl2")
    public Client client2() {
        Client resu = new Client();
        resu.setNom("Dupont");
        return resu;
    }
}
```

Définition Simple - Classe annotée

- Plutôt que de demander au contexte Spring de charger vos classes de configuration, vous pouvez faire des liens entre elles via l'annotation `@org.springframework.context.annotation.Import`
- Indiquez simplement la *class* que vous voulez importer
- Remarque : cette technique **n'est pas** à privilégier

```
@Configuration
public class SpringConfigurationA {
    @Bean
    public Client client1() {
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }
}
```

```
@Configuration
@Import(SpringConfigurationA.class)
public class SpringConfigurationB {
    @Bean
    public Client client2() {
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }
}
```

Définition Simple - Classe annotée

- Vous pouvez aussi importer une configuration XML dans votre classe de configuration via l'annotation `@org.springframework.context.annotation.ImportResource`
- Indiquez simplement le/les fichier(s) XML que vous voulez importer
- Remarque : cette technique **n'est pas** à privilégier

```
@Configuration
@ImportResource({"classpath*:monfichier.xml"})
public class SpringConfigurationB {
    @Bean
    public Client client2() {
        Client resu = new Client();
        resu.setNom("Dupont");
        resu.setPrenom("Jean");
        resu.setAge(45);
        return resu;
    }
}
```

Définition Simple - Classe annotée

➤ Remarques :

- **L'ordre** de déclaration de vos méthodes **@Bean** n'a aucune importance !
- **L'ordre** de vos paramètres dans vos méthodes **@Bean** n'a aucune importance !
- **L'ordre** de vos annotations sur la classe de configuration n'a aucune importance !



Travaux pratiques

17

Réaliser les travaux pratiques suivants:

TP 01 : Mise en place d'un projet Spring dans Eclipse

TP 02 : Création de beans

TP 03 : Utilisation des List

Création d'un contexte applicatif

Présentation (1/2)

- Les contextes applicatifs Spring peuvent être initialisés dans n'importe quel environnement :
 - Test Unitaires Automatisés JUnit.
 - Application Web.
 - Enterprise Java Bean (EJB).
 - Application autonome (Swing, Batch).

Création d'un contexte applicatif

Présentation (2/2)

- L'interface la plus utilisée est `org.springframework.context.ApplicationContext`
- Spring propose plusieurs implémentations dont, entre autres :
 - `org.springframework.context.annotation.AnnotationConfigApplicationContext`
 - ❑ Chargement du contexte à partir de classes annotées
 - `org.springframework.web.context.support.AnnotationConfigWebApplicationContext`
 - ❑ Chargement de contexte à partir de classes annotées au sein d'une application Web.

Création d'un contexte applicatif

Modes de chargement

- Spring offre un large éventail de possibilités pour créer un contexte applicatif

- Via les classes annotées en `@Configuration`

```
new AnnotationConfigApplicationContext(MaClasse.class);
```

- Via un nom de package (qui contiendra des classes annotées)

```
new AnnotationConfigApplicationContext("com.monprj.springconfigpkg");
```

Création d'un contexte applicatif

Bonnes Pratiques

- Un contexte Spring peut être initialisé à partir de plusieurs classes annotées en `@Configuration`
- Permet de séparer la définition des beans en les regroupant par thématiques
 - Domaine métier
 - Accès à la base de données
 - Web services
 - etc...
- Il est conseillé d'extraire la configuration des beans métiers de ceux relatifs à l'environnement
 - Faciliter le déploiement en recette, intégration et production
 - Permet l'utilisation des Tests Unitaires Automatisés

Création d'un contexte applicatif

Bonnes Pratiques

- Par exemple, en test, le serveur de base de données est MySQL et en production Oracle

- Création du contexte applicatif en environnement de test

```
ApplicationContext context = new AnnotationConfigApplicationContext(  
    ConfigurationBO.class,  
    ConfigurationDataBaseMySQL.class);
```

- Création du contexte applicatif en environnement de production

```
ApplicationContext context = new AnnotationConfigApplicationContext(  
    ConfigurationBO.class,  
    ConfigurationDataBaseOracle.class);
```

- Vous pouvez séparer les classes par une **virgule**

Les Profiles

➤ **@Profile** permet de définir le profil qui rendra le bean existant

```
@Configuration
public class SpringConfiguration {
    @Bean
    @org.springframework.context.annotation.Profile("test")
    public DataSource dataSourcePourLeTest() {
        BasicDataSource resu = new org.apache.commons.dbcp2.BasicDataSource();
        resu.setDriverClassName("com.mysql.cj.jdbc.Driver");
        resu.setUrl("jdbc:mysql://localhost/banque");
        resu.setUsername("root");
        resu.setPassword("root");
        return resu;
    }

    @Bean
    @org.springframework.context.annotation.Profile("production")
    public DataSource dataSourcePourLaProduction() throws NamingException {
        JndiTemplate jndi = new JndiTemplate();
        return jndi.lookup("java:comp/env/jdbc/netbankPool", DataSource.class);
    }
}
```

N'existe que
pour le profil
test

N'existe que
pour le profil
prod

Les Profiles

- **@Profile** fonctionne aussi sur les classes @Configuration
- Elle peut aussi porter plusieurs valeurs
 - Pratique pour les beans appartenant à plusieurs profils

```
@Configuration
@Profile({"test", "dev"})
public class SpringConfigurationPourLeTest {
    @Bean
    public DataSource dataSourcePourLeTest() {
        ...
    }

    @Bean
    ...
}
```

Tous les beans de cette classe n'existent que pour le profil *test* et le *dev*

Les Profiles

- Lors du chargement du contexte vous précisez le/les profil(s) que vous voulez
- **Attention** : cela change la mécanique de chargement

```
AnnotationConfigApplicationContext appContext = new AnnotationConfigApplicationContext();  
appContext.getEnvironment().setActiveProfiles("test");  
appContext.register(SpringConfiguration.class);  
appContext.refresh();
```

```
DataSource ds = appContext.getBean(DataSource.class);
```

- On peut aussi appeler **addActiveProfile**
 - Afin d'ajouter un profile

Récupération d'instances de bean

- Vous pouvez récupérer vos beans à travers la méthode **getBean** de votre objet contexte
- Récupération d'un bean avec transtypage (« cast »)

```
ITransfertService service = (ITransfertService) ctx.getBean("transfertService");  
service.transfert(2956);
```

- L'instance d'un bean est récupérée du contexte par la valeur de l'attribut « name » (identifiant unique) de l'annotation **@Bean**.

- Depuis la version 3 de Spring, il est possible de récupérer un bean sans cast

```
ITransfertService service = ctx.getBean("transfertService", ITransfertService.class);  
  
// Il n'est pas nécessaire de préciser son identifiant si le bean est unique.  
ITransfertService service = ctx.getBean(ITransfertService.class);
```

Prise en main de Spring

Conclusion

- Spring instancie les beans dans l'ordre dicté par les dépendances entre beans
 - Les beans peuvent être déclarés suivant un ordre quelconque
 - Spring se charge automatiquement de l'ordonnement.
- Chaque bean se voit affecter un identifiant unique par contexte
- Spring centralise la gestion du cycle de vie des instances d'objets

Configuration via un fichier *properties*

- **Objectif** : externaliser des paramètres de configuration dans des fichiers propriétés
- Pour en faire usage directement dans les classes de configuration mais aussi dans nos classes gérées par Spring
- On utilisera les annotations :
 - `@org.springframework.context.annotation.PropertySource`
 - `@org.springframework.context.annotation.PropertySources`
 - `@org.springframework.context.annotation.Value`

Configuration via un fichier *properties*

- `@org.springframework.context.annotation.PropertySource` se placera sur la classe de configuration et chargera le fichier de *properties* en mémoire

```
@Configuration
@PropertySource("classpath:spring/adresses.properties")
public class SpringConfiguration {
```

- Si vous avez plusieurs fichiers *properties* à charger, faites usage de `@org.springframework.context.annotation.PropertySources`

```
@Configuration
@PropertySources({
    @PropertySource("classpath:spring/adresses.properties"),
    @PropertySource("classpath:spring/adresses2.properties")
})
public class SpringConfiguration {
```

Configuration via un fichier *properties*

- Vous pouvez faire usage de vos propriétés de trois manières différentes
 - Via **@Value** sur un attribut
 - Via **@Value** sur un paramètre
 - Via l'interface *Environment* du Spring et son injection via un **@Autowired**

Configuration via un fichier *properties*

➤ Remarques :

- Spring fait usage d'un objet de type **PropertyPlaceholderConfigurer**
- Le fichier *properties* est chargé dans le contexte Spring et cette opération est faite avant le chargement de vos beans
- Vous pouvez utiliser toutes les clefs du fichier *properties*
 - ❑ **ATTENTION** : aux majuscules / minuscules, aux noms de vos clefs
 - Faites copier/coller de vos noms de clefs

Configuration via un fichier *properties*

➤ Exemple avec @Value

```
@Configuration
@PropertySource("classpath:info.properties")
public class SpringConfiguration {
    @Value("${datasource.url}")
    private String dbUrl;

    @Bean
    public AccessBD monDaoLike(@Value("${datasource.login}") String pLogin) {
        AccessBD bd = new AccessBD();
        bd.setUrl(this.dbUrl);
        bd.setLogin(pLogin);
    }
}
```

```
datasource.url=jdbc:mysql://localhost/banque?useSSL=false
datasource.login=root
```

Fichier info.properties

Configuration via un fichier *properties*

➤ Exemple avec l'interface *Environment* du Spring

```
@Configuration
@PropertySource("classpath:info.properties")
public class SpringConfiguration {
    @Autowired
    private Environment env;

    @Bean
    public AccessBD monDaoLike() {
        AccessBD bd = new AccessBD();
        // ATTENTION : pas de ${..} ici
        bd.setUrl(this.env.getProperty("datasource.url"));
        bd.setLogin(this.env.getProperty("datasource.login"));
    }
}
```

Demande au
Spring d'injecter
l'objet ici, dans
l'attribut env

```
datasource.url=jdbc:mysql://localhost/banque?useSSL=false
datasource.login=root
```

Fichier *info.properties*



Travaux pratiques

34

Réaliser les travaux pratiques suivants:

TP 04 : Utilisation d'un fichier properties

Cycle de vie du contexte applicatif

Travailler avec les post-processeurs - BeanPostProcessor

- Par défaut, Spring instancie les beans lors du démarrage du Contexte Applicatif
 - L'instanciation est réalisée dans l'ordre et toutes les dépendances sont injectées.
- Dès l'injection des dépendances réalisée, la phase de post traitement peut débuter
 - Les actions de configuration plus poussées sont réalisées.
- Après la phase de post-traitement, le bean est complètement initialisé et prêt à l'emploi
 - Chaque bean est utilisé jusqu'à sa destruction.

Cycle de vie du contexte applicatif

Travailler avec les post-processeurs - BeanPostProcessor

- « BeanPostProcessor » est un point d'extension important
 - Permet de modifier une instance d'un bean après son instantiation par le conteneur.
- Spring fournit un certain nombre d'implémentations
 - Il est possible mais assez rare d'écrire soi-même une implémentation.
- Exemples d'implémentation
 - RequiredAnnotationBeanPostProcessor
 - ❑ Fourni par Spring.
 - ❑ Assure que les attributs annotés par « @Required » sont peuplés.

Cycle de vie du contexte applicatif

Initialisation – Approche JSR

- Le post-traitement est réalisable via les annotations JSR-250

```
@PostConstruct  
public void init() {  
    // Code applicatif d'initialisation  
}
```

- Donne la consigne à Spring d'appeler la méthode après injection.
 - Aucune contrainte sur la visibilité ou le nom appliqué à la méthode.
- Cette méthode est conseillée
 - Pas d'adhérence au framework Spring.
 - Conforme aux spécifications JSR-250 donc portable.

Cycle de vie du contexte applicatif

Initialisation – Approche Annotation ou Interface

➤ Activation via la déclaration du `@Bean`

- Exécutera la méthode *init()* de l'objet *AccessBD* automatiquement après la méthode *monDaoLike()*.

```
@Configuration
public class SpringConfiguration {

    @Bean(initMethod="init")
    public AccessBD monDaoLike() {
        // ...
    }
}
```

- ### ➤ **OU** : faire implémenter l'interface « `org.springframework.beans.factory.InitializingBean` » à l'objet visé (ici *AccessBD*)

Cycle de vie du contexte applicatif

Phase de destruction (1/2)

- Le Contexte Applicatif est stoppé
 - Destruction explicite des beans selon la configuration indiquée.
 - Destruction du Contexte Applicatif ; il n'est plus utilisable.
- Un bean peut implémenter des « callback » qui sont appelés lors de la phase de destruction
 - Utilisé pour libérer des ressources
 - ❑ Pointeurs vers des fichiers
 - ❑ Sockets
- Par annotation « **@PreDestroy** »
 - Standard JSR-250

```
@PreDestroy  
public void stop() {  
    // Code applicatif de destruction  
}
```

Cycle de vie du contexte applicatif

Phase de destruction (2/2)

➤ Activation via la déclaration du `@Bean`

- Exécutera la méthode `stop()` de l'objet `AccessBD` automatiquement après l'arrêt du contexte Spring.

```
@Configuration
public class SpringConfiguration {

    @Bean(destroyMethod="stop")
    public AccessBD monDaoLike() {
        // ...
    }
}
```

- ### ➤ **OU** : faire implémenter l'interface
- « `org.springframework.beans.factory.DisposableBean`
 - » à l'objet visé (ici `AccessBD`)



Travaux pratiques

41

Réaliser les travaux pratiques suivants:

TP 05 : Init et Destroy

Cycle de vie du contexte applicatif

Portées des beans Spring

- Spring définit pour chaque bean sa portée (son scope)
- Les portées disponibles
 - **singleton** une seule instance par contexte. C'est la portée par défaut.
 - **prototype** nouvelle instance créée chaque fois que le bean est injecté.
 - **session** nouvelle instance créée par session utilisateur. Environnement Web.
 - **request** nouvelle instance créée pour chaque requête. Environnement Web.
 - **custom** définition de nouvelles règles de gestion. Programmation avancée.
- Configuration de la portée dans la définition du bean via l'annotation `@org.springframework.context.annotation.Scope`

Cycle de vie du contexte applicatif

Portées des beans Spring

➤ Exemple

```
@Configuration
public class SpringConfiguration {

    @Bean
    @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
    public AccessBD monDaoLike() {
        // ...
    }
}
```



Travaux pratiques

44

Réaliser les travaux pratiques suivants:

TP 06 : Utilisation des Scope

TP 07 : Un projet complet

Annotations conditionnelles

- Spring met à disposition des annotations conditionnelles génériques (plus spécifiques en Spring Boot)
- Avec, vous pouvez dire de charger tel bean si tel autre est présent ou absent par exemple
- Pratique quand on fait usage des profils

Annotations conditionnelles

- Sur votre `@Bean` ou `@Configuration` faites usage de l'annotation **`@Conditional`**
- Cette dernière fera usage d'une classe qui implémentera l'interface **`Condition`** du Spring (ou **`ConfigurationCondition`** si vous voulez plus de granularité sur les conditions de configuration)
 - Elle implémentera la méthode **`boolean matches(ConditionContext, AnnotatedTypeMetadata)`**

Annotations conditionnelles

47

➤ Exemple d'utilisation @Conditional

```
public class LocalConfigCondition implements Condition {  
    public boolean matches(ConditionContext conditionContext, AnnotatedTypeMetadata annotatedTypeMetadata) {  
        return "local".equals(conditionContext.getEnvironment().getProperty("config.condition", "local"));  
    }  
}
```

```
public class RemoteConfigCondition implements Condition {  
    public boolean matches(ConditionContext conditionContext, AnnotatedTypeMetadata annotatedTypeMetadata) {  
        return "remote".equals(conditionContext.getEnvironment().getProperty("config.condition"));  
    }  
}
```

```
...  
@Bean  
@Conditional(LocalConfigCondition.class)  
public IFaireQQc faireLocal() {  
    return new MonBeanA();  
}  
...
```

```
...  
@Bean  
@Conditional(RemoteConfigCondition.class)  
public IFaireQQc faireRemote() {  
    return new MonBeanB();  
}  
...
```